

04. 분류 - Part2

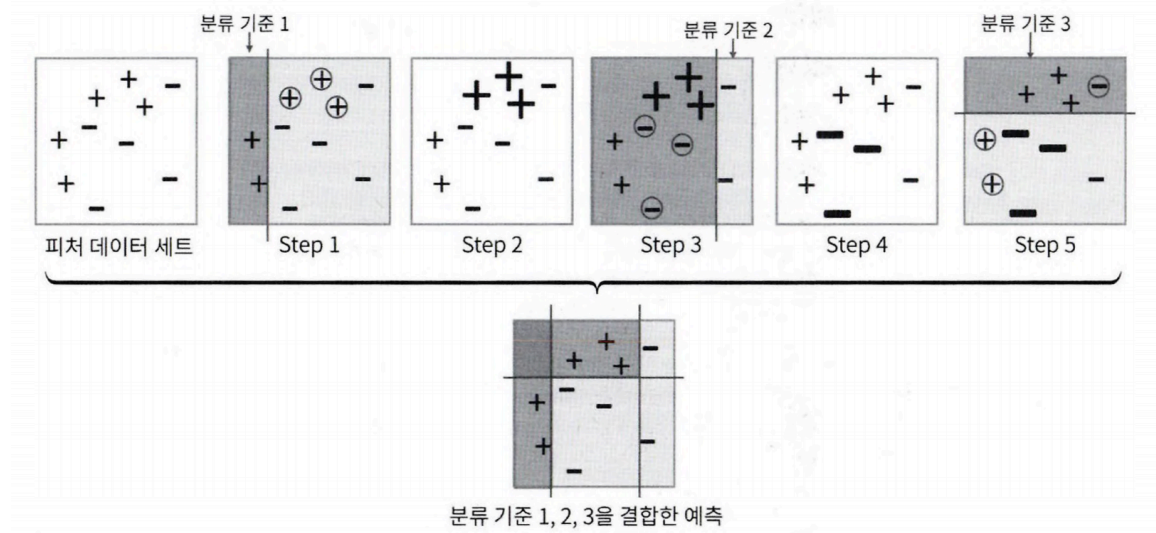
05. GBM(Gradient Boosting Machine)

GBM의 개요 및 실습

부스팅 알고리즘은 여러 개의 약한 학습기를 순차적으로 학습 - 예측하면서 잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식

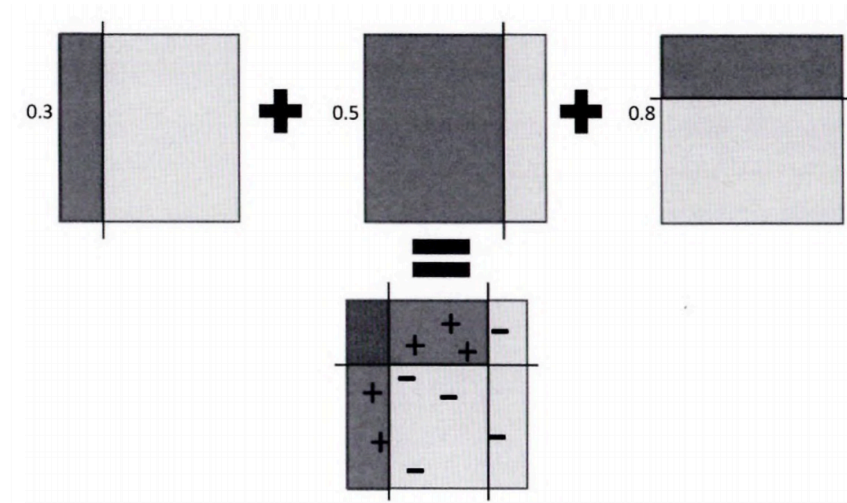
- 대표적인 구현은 AdaBoost(Adaptive boosting)와 그래디언트 부스트가 있음
- 에이다 부스트는 오류 데이터에 가중치를 부여하면서 부스팅을 수행하는 대표적인 알고리즘

AdaBoost(에이다부스트)



- Step 1: 첫 번째 약한 학습기가 분류 기준 1로 +와 -를 분류한 것. 동그라미로 표시된 +데이터는 +데이터가 잘못 분류된 오류 데이터
- Step 2: 이 오류 데이터에 대해서 가중치 값을 부여. 가중치가 부여된 오류 +데이터는 다음 약한 학습기가 더 잘 분류할 수 있게 크기가 커짐

- Step 3: 두 번째 약한 학습기가 분류 기준 2로 +와 -를 분류. 마찬가지로 동그라미로 표시된 - 데이터는 잘못 분류된 오류 데이터
- Step 4: 잘못 분류된 이 - 오류 데이터에 대해 다음 약한 학습기가 잘 분류할 수 있게 더 큰 가중치를 부여
- Step 5: 세 번째 약한 학습기가 분류 기준 3으로 +와 -를 분류하고 오류 데이터를 찾음. 에이다부스트는 이렇게 약한 학습기가 순차적으로 오류 값에 대해 가중치를 부여한 예측 결정 기준을 모두 결합해 예측을 수행
- 맨 아래에는 첫 번째, 두 번째, 세 번째 약한 학습기를 모두 결합한 결과 예측. 개별 약한 학습기보다 훨씬 정확도가 높아졌음을 알 수 있음



↳ 개별 약한 학습기는 다음 그림과 같이 각각 가중치를 부여해 결합함. 예를 들어 첫 번째 학습기에 가중치 0.3, 두 번째 학습기에 가중치 0.5, 세 번째 학습기에 가중치 0.8을 부여한 후 모두 결합해 예측을 수행

GBM(Gradient Boost Machine)

GBM은 AdaBoost와 달리 가중치 업데이트를 경사 하강법을 이용함

? 경사 하강법(Gradient Decent)

- 반복 수행을 통해 오류를 최소화할 수 있도록 가중치의 업데이트 값을 도출하는 기법

- 오류 값 = 실제 값 - 예측 값
- 분류의 실제 결괏값은 y , 피처를 $x_1, x_2, \dots, x_N$ 그리고 이 피처에 기반한 예측 함수를 $F(x)$ 함수라고 하면 오류식은 $h(x) = y - F(x)$ 가 됨
- 이 오류식을 최소화하는 방향성을 가지고 반복적으로 가중치 값을 업데이트 하는 것이 경사하강법

GBM은 CART 기반의 다른 알고리즘과 마찬가지로 분류, 회귀 모두 가능

일반적으로 GBM이 랜덤 포레스트보다는 예측 성능이 조금 뛰어난 경우가 많음. 그러나 수행 시간이 오래 걸리고, 하이퍼 파라미터 튜닝 노력도 더 필요함

GBM 하이퍼 파라미터 소개

- **loss**: 경사 하강법에서 사용할 비용 함수를 지정. 특별한 이유가 없으면 기본값인 'deviance'를 그대로 적용
- **learning_rate**: GBM이 학습을 진행할 때마다 적용하는 학습률. Weak learner가 순차적으로 오류 값을 보정해 나가는 데 적용하는 계수. 0~1 사이의 값을 지정할 수 있으며 기본값은 0.1. 너무 작은 값을 적용하면 업데이트되는 값이 작아져서 최소 오류 값을 찾아 예측 성능이 높아질 가능성이 높음. 하지만 많은 weak learner는 순차적인 반복이 필요해서 수행 시간이 오래 걸리고, 또 너무 작게 설정하면 모든 weak learner의 반복이 완료돼도 최소 오류 값을 찾지 못할 수 있음. 이러한 특성 때문에 learning_rate는 n_estimators와 상호 보완적으로 조합해 사용. learning_rate를 작게 하고 n_estimator를 크게 하면 더 이상 성능이 좋아지지 않는 한계점까지 예측 성능이 조금씩 좋아질 수 있음. 하지만 수행 시간이 너무 오래 걸리는 단점이 있고, 예측 성능 역시 현격히 좋아지지는 않음
- **n_estimator**: weak learner의 개수. weak learner가 순차적으로 오류를 보정하므로 개수가 많을수록 예측 성능이 일정 수준까지는 좋아질 수 있으나 개수가 많을수록 수행 시간이 오래 걸림. 기본값은 100.
- **subsample**: weak learner가 학습에 사용하는 데이터의 샘플링 비율. 기본값은 1. 이는 전체 학습 데이터를 기반으로 학습한다는 의미. 과적합이 염려되는 경우 subsample을 1보다 작은 값으로 설정

06. XGBoost(eXtra Gradient Boost)

| XGBoost 개요

XGBoost는 트리 기반의 앙상블 학습에서 가장 주목받고 있는 알고리즘 중 하나

XGBoost는 GBM에 기반하고 있지만, GBM의 단점인 느린 수행 시간 및 과적합 규제 (regularization) 부재 등의 문제를 해결해서 매우 각광받고 있음. 특히 XGBoost는 병렬 CPU 환경에서 병렬 학습이 가능해 기존 GBM보다 빠르게 학습을 완료할 수 있음.

항목	설명
뛰어난 예측 성능	일반적으로 분류와 회귀 영역에서 뛰어난 예측 성능을 발휘합니다.
GBM 대비 빠른 수행 시간	일반적인 GBM은 순차적으로 Weak learner가 가중치를 증감하는 방법으로 학습하기 때문에 전반적으로 속도가 느립니다. 하지만 XGBoost는 병렬 수행 및 다양한 기능으로 GBM에 비해 빠른 수행 성능을 보장합니다. 아쉽게도 XGBoost가 일반적인 GBM에 비해 수행 시간이 빠르다는 것이지, 다른 머신러닝 알고리즘(예를 들어 랜덤 포레스트)에 비해서 빠르다는 의미는 아닙니다.
과적합 규제 (Regularization)	표준 GBM의 경우 과적합 규제 기능이 없으나 XGBoost는 자체에 과적합 규제 기능으로 과적합에 좀 더 강한 내구성을 가질 수 있습니다.
Tree pruning (나무 가지치기)	일반적으로 GBM은 분할 시 부정 손실이 발생하면 분할을 더 이상 수행하지 않지만, 이러한 방식도 자칫 지나치게 많은 분할을 발생할 수 있습니다. 다른 GBM과 마찬가지로 XGBoost도 max_depth 파라미터로 분할 깊이를 조정하기도 하지만, tree pruning으로 더 이상 긍정 이득이 없는 분할을 가지치기 해서 분할 수를 더 줄이는 추가적인 장점을 가지고 있습니다.
자체 내장된 교차 검증	XGBoost는 반복 수행 시마다 내부적으로 학습 데이터 세트와 평가 데이터 세트에 대한 교차 검증을 수행해 최적화된 반복 수행 횟수를 가질 수 있습니다. 지정된 반복 횟수가 아니라 교차 검증을 통해 평가 데이터 세트의 평가 값이 최적화 되면 반복을 중간에 멈출 수 있는 조기 중단 기능이 있습니다.
결손값 자체 처리	XGBoost는 결손값을 자체 처리할 수 있는 기능을 가지고 있습니다.

└ XGBoost의 주요 장점

파이썬 래퍼 XGBoost 하이퍼 파라미터

GBM과 유사한 하이퍼 파라미터를 동일하게 가지고 있으며, 여기에 조기 중단(early stopping), 과적합을 규제하기 위한 하이퍼 파라미터 등이 추가됨

파이썬 래퍼 XGBoost 모듈과 사이킷런 래퍼 XGBoost 모듈의 일부 하이퍼 파라미터는 약간 다르므로 주의가 필요함.

- 동일한 기능을 의미하는 하이퍼 파라미터이지만, 사이킷런 파라미터의 범용화된 이름 규칙에 따라 파라미터 명이 달라짐.
- **일반 파라미터**: 일반적으로 실행 시 스레드의 개수나 silent 모드 등의 선택을 위한 파라미터로서 디폴트 파라미터 값을 바꾸는 경우는 거의 없음
- **부스터 파라미터**: 트리 최적화, 부스팅, regularization 등과 관련 파라미터 등을 지칭
- **학습 태스크 파라미터**: 학습 수행 시의 객체 함수, 평가를 위한 지표 등을 설정하는 파라미터
- 대부분의 하이퍼 파라미터는 Booster 파라미터에 속함

▼ 주요 일반 파라미터

booster: gbtree(tree based model) 또는 gblinear(linear model) 선택, 디폴트는 gbtree

silent: 디폴트는 0, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정

nthread: CPU의 실행 스레드 개수를 조정, 디폴트는 CPU의 전체 스레드를 다 사용하는 것. 멀티 코어/스레드 CPU 시스템에서 전체 CPU를 사용하지 않고 일부 CPU만 사용해 ML 애플리케이션을 구동하는 경우에 변경

▼ 주요 부스터 파라미터

eta [default=0.3, alias: learning_rate]: GBM의 학습률과 같은 파라미터. 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 파이썬 래퍼 기반의 xgboost를 이용할 경우 디폴트는 0.3. 사이킷런 래퍼 클래스를 이용할 경우 eta는 learning_rate 파라미터로 대체되며, 디폴트는 0.1. 보통은 0.01~0.2 사이의 값을 선호

num_boost_rounds: GBM의 n_estimators와 같은 파라미터

min_child_weight [default=1]: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 weight 총합. 해당 파라미터가 클수록 분할을 자제. 과적합을 조절하기 위해 사용

gamma [default=0, alias: min_spilt_loss]: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값. 해당 값보다 큰 손실이 감소된 경우에 리프 노드를 분리. 값이 클수록 과적합 감소 효과가 있음

max_depth [default=6]: 트리 기반 알고리즘의 max_depth와 같음. 0을 지정하면 깊이에 제한이 없음. Max_depth가 높으면 특정 피쳐 조건에 특화되어 룰 조건이 만들어 지므로 과적합 가능성이 높아지며 보통은 3~10 사이의 값을 적용

sub_sample [default=1]: GBM의 subsample과 동일. 트리가 커져서 과적합되는 것을 제어하기 위해 데이터를 샘플링하는 비율을 지정. sub_sample=0.5로 지정하면 전체 데이터의 절반을 트리를 생성하는데 사용. 0에서 1사이의 값이 가능하나 일반적으로 0.5~1 사이의 값을 사용

colsample_bytree [default=1]: GBM의 max_features와 유사. 트리 생성에 필요한 피쳐를 임의로 샘플링하는 데 사용됨. 매우 많은 피쳐가 있는 경우 과적합을 조정하는데 적용

lambda [default=1, alias: reg_lambda]: L2 Regularization 적용 값. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있음

alpha [default=0, alias: reg_alpha]: L1 Regularization 적용 값. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있음

scale_pos_weight [default=1]: 특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터

▼ 학습 태스크 파라미터

objective: 최솟값을 가져야 할 손실 함수를 정의. XGBoost는 많은 유형의 손실함수를 사용할 수 있음. 주로 사용되는 손실함수는 이진 분류인지 다중 분류인지에 따라 달라짐

binary:logistic: 이진 분류일 때 적용

multi:softmax: 다중 분류일 때 적용. 손실함수가 multi:softmax일 경우에는 레이블 클래스의 개수인 num_class 파라미터 지정해야 함

multi:softprob: mutli_softmax와 유사하나 개별 레이블 클래스의 해당되는 예측 확률을 반환

eval_metric: 검증에 사용되는 함수를 정의. 기본값은 회귀인 경우는 rmse, 분류일 경우에는 error. 다음은 eval_metirc의 값 유형

- rmse: Root Mean Square Error
- mae: Mean Absolute Error
- logloss: Negative log_likelihood

- error: Binary classification error rate (0.5 threshold)
- merror: Multiclass classification error rate
- mlogloss: Multiclass logloss
- auc: Area under the curve

사이킷런 래퍼 XGBoost의 개요 및 적용

사이킷런의 기본 Estimator를 그대로 상속해 만들었기 때문에 다른 Estimator와 동일하게 `fit()`와 `predict()`만으로 학습과 예측이 가능. `GridSearchCV`, `Pipeline` 등 사이킷런의 다른 유틸리티를 그대로 사용할 수 있기 때문에 기존의 다른 머신러닝 알고리즘으로 만들어놓은 프로그램이 있더라도 알고리즘 클래스만 XGBoost 래퍼 클래스로 바꾸면 기존 프로그램을 그대로 사용 가능

▼ `XGBClassifier`는 기존 사이킷런에서 일반적으로 사용하는 하이퍼 파라미터와 호환성을 유지하기 위해 기존의 `xgboost` 모듈에서 사용하던 네이티브 하이퍼 파라미터 몇 개를 다음과 같이 변경함

`eta` → `learning_rate`

`sub_sample` → `subsample`

`lambda` → `reg_lambda`

`alpha` → `reg_alpha`

- `xgboost`의 `n_estimator`와 `num_boost_round` 하이퍼 파라미터는 서로 동일한 파라미터. 만일 두 개가 동시에 사용되면 파이썬 래퍼 XGBoost API에서는 `n_estimators` 파라미터를 무시하고 `num_boost_round` 파라미터를 적용. 하지만 `XGBClassifier`와 같은 사이킷런 래퍼 XGBoost 클래스에서는 `n_estimators` 파라미터를 적

07. LightGBM

LightGBM은 XGBoost와 함께 부스팅 계열 알고리즘에서 가장 각광 받고 있음. Xgboost는 매우 뛰어난 부스팅 알고리즘이지만, 여전히 학습 시간이 오래 걸림.

XGBoost에서 GridSearchCV로 하이퍼 파라미터 튜닝을 수행하다 보면 수행 시간이 너무 오래 걸려서 많은 파라미터를 튜닝하기에 어려움을 겪을 수 있음.

? LightGBM의 가장 큰 장점은 XGBoost보다 학습에 걸리는 시간이 훨씬 적고, 메모리 사용량도 상대적으로 적음

- LightGBM과 XGBoost의 예측 성능은 별다른 차이가 없음
- 기능상 다양성은 LightGBM이 약간 더 많음

LightGBM의 한 가지 단점은 적은 데이터 세트에 적용할 경우 과적합이 발생하기 쉽다는 점

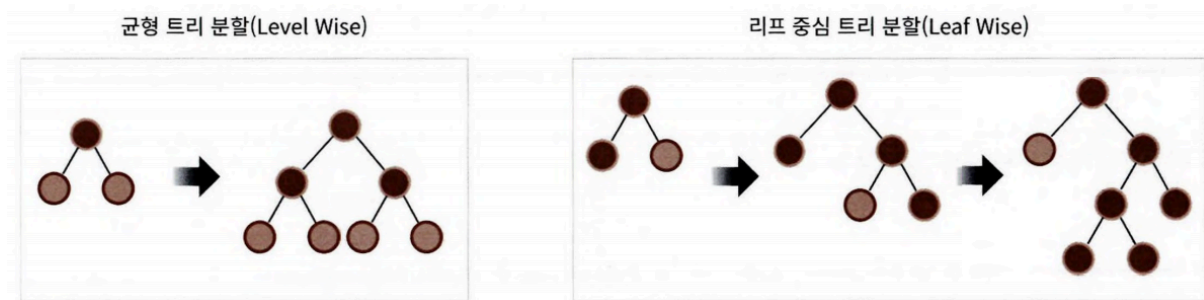
LightGBM은 일반 GBM계열의 트리 분할 방법과 다르게 리프 중심 트리 분할 방식을 사용

균형 트리 분할(Level Wise)

- 트리의 깊이를 효과적으로 줄이기 위함
- 최대한 균형 잡힌 트리를 유지하면서 분할하기 때문에 트리의 깊이가 최소화 될 수 있음
- 균형 잡힌 트리는 오버피팅에 보다 더 강한 구조를 가짐
- 균형을 맞추기 위한 시간이 필요함

리프 중심 트리 분할(Leaf Wise)

- 트리의 균형을 맞추지 않음
- 최대 손실 값(max delta loss)을 가지는 리프 노드를 지속적으로 분할하면서 트리가 깊어지고 비대칭적인 규칙 트리가 생성됨
- 최대 손실값을 가지는 리프 노드를 지속적으로 분할해 생성된 규칙 트리는 학습을 반복할수록 결국은 균형 트리 분할 방식보다 예측 오류 손실을 최소화 할 수 있다는 것이 LightBGM의 구현 사상



LightGBM의 XGBoost 대비 장점

- 더 빠른 학습과 예측 수행 시간
- 더 작은 메모리 사용량
- 카테고리형 피처의 자동 변환과 최적 분할(원-핫 인코딩 등을 사용하지 않고도 카테고리형 피처를 최적으로 변환하고 이에 따른 노드 분할 수행)

LightGBM 하이퍼 파라미터

LightGBM 하이퍼 파라미터는 XGBoost와 많은 부분이 유사. 주의할 점은 LightGBM은 XGBoost와 다르게 리프 노드가 계속 분할되면서 트리 깊이가 깊어지므로 이러한 트리 특성에 맞는 하이퍼 파라미터 설정이 필요하다는 점

▼ 주요 파라미터

num_iterations [default=100]: 반복 수행하려는 트리의 개수를 지정. 크게 지정할수록 예측 성능이 높아질 수 있으나, 너무 크게 지정하면 오히려 과적합으로 성능이 저하될 수 있음. 사이킷런 GBM과 XGBoost의 사이킷런 호환 클래스의 `n_estimators`와 같은 파라미터이므로 LightGBM의 사이킷런 호환 클래스에서는 `n_estimators`로 이름이 변경됨

learning_rate [default=0.1]: 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 일반적으로 `n_estimators`를 크게 하고 `learning_rate`를 작게 해서 예측 성능을 향상시킬 수 있으나, 마찬가지로 과적합 문제와 학습 시간이 기어지는 부정적인 영향도 고려해야 함. GBM, XGBoost의 Learning rate와 같은 파라미터

max_depth [default=-1]: 트리 기반 알고리즘의 max_depth와 같음. 0보다 작은 값을 지정하면 깊이에 제한이 없음. 지금까지의 Depth wise 방식의 트리와 다르게 LightGBM은 Leaf wise 기반이므로 깊이가 상대적으로 더 깊음

min_data_in_leaf [default=31]: 하나의 트리가 가질 수 있는 최대 리프 개수

boosting [default=gbdt]: 부스팅의 트리를 생성하는 알고리즘을 기술

- **gbdt:** 일반적인 그래디언트 부스팅 결정 트리
- **rf:** 랜덤 포레스트

bagging_fraction [default=1.0]: 트리가 커져서 과적합되는 것을 제어하기 위해서 데이터를 샘플링하는 비율을 지정. 사이킷런의 GBM과 XGBClassifier의 subsample 파라미터와 동일하기에 사이킷런 래퍼 LightGBM인 LightGBMClassifier에서는 subsample로 동일하게 파라미터 이름이 변경됨

feature_fraction [default=1.0]: 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율. 과적합을 막기 위해 사용됨. GBM의 max_features와 유사, XGBClassifier의 colsample_bytree와 똑같으므로 LightGBMClassifier에서는 동일하게 colsample_bytree로 변경됨

lambda_l2 [default=0.0]: L2 regulation 제어를 위한 값, 피처 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있음. XGBClassifier의 reg_lambda와 동일하므로 LightGBMClassifier에서는 reg_lambda로 변경됨

lambda_l1 [default=0.0]: L1 Regulation 제어를 위한 값. L2와 마찬가지로 과적합 제어를 위한 것이며, XGBClassifier의 reg_alpha와 동일하므로 LightGBMClassifier에서는 reg_alpha로 변경됨

▼ Learning Task 파라미터

objective: 최솟값을 가져야 할 손실함수를 정의. Xgboost의 objective 파라미터와 동일. 애플리케이션 유형, 즉 회귀, 다중 클래스 분류, 이진 분류인지에 따라서 objective인 손실함수가 지정됨

| 하이퍼 파라미터 튜닝 방안

num_leaves의 개수를 중심으로 min_child_samples(min_data_in_leaf), max_depth를 함께 조정하면서 모델의 복잡도를 줄이는 것이 기본 튜닝 방안

- num_leaves는 개별 트리가 가질 수 있는 최대 리프 개수
- min_data_in_leaf는 사이킷런 래퍼 클래스에서는 min_child_samples로 이름이 변경됨. 과적합을 개선하기 위한 중요한 파라미터.
- max_depth는 명시적으로 깊이의 크기를 제한함. num_leaves, min_data_in_leaf와 결합해 과적합을 개선하는 데 사용함

learning_rate를 작게 하면서 n_estimators를 크게 하는 것은 부스팅 계열 튜닝에서 가장 기본적인 튜닝 방안이므로 이를 적용하는 것도 좋음. 그러나 n_estimators를 너무 크게 하는 것은 과적합으로 오히려 성능이 저하될 수 있음

이 밖에 과적합을 제어하기 위해서 reg_lambda, reg_alpha와 같은 regularization을 적용하거나 학습 데이터에 사용할 피처의 개수나 데이터 샘플링 레코드 개수를 줄이기 위해 colsample_bytree, subsample 파라미터를 적용할 수 있음

파이썬 래퍼 LightGBM과 사이킷런 래퍼 XGBoost, LightGBM 하이퍼 파라미터 비교

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

08. 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

지금까지 하이퍼 파라미터 튜닝을 위해서 사이킷런에서 제공하는 Grid Search 방식을 적용함. Grid Search 방식은 한 가지 주요한 단점이 있는데, 튜닝해야 할 하이퍼 파라미터 개수가 많을 경우 최적화 수행 시간이 오래 걸린다는 것.

XGBoost나 LightGBM은 성능이 매우 뛰어난 알고리즘이지만, 하이퍼 파라미터 개수가 다른 알고리즘에 비해 많음. 때문에 실무의 대용량 학습 데이터에 Grid Search 방식으로 최적 하이퍼 파라미터를 찾으려면 많은 시간이 소모될 수 있음

⇒ 실무의 대용량 학습 데이터에 XGBoost나 LightGBM의 하이퍼 파라미터 튜닝 시에 Grid Search 방식보다는 다른 방식을 적용하곤 하는데 대표적으로 베이지안 최적화 기법이 있음

베이지안 최적화 개요

베이지안 최적화는 목적 함수 식을 제대로 알 수 없는 블랙 박스 형태의 함수에서 최대 또는 최소 함수 반환 값을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식

? 함수 $f(x, y) = 2x - 3y$ 가 있다고 하면 $f(x, y)$ 의 반환 값을 최대/최소로 하는 x, y 값을 찾아내는 것. 위의 함수는 간단한 함수이므로 쉽게 반환값을 최대/최소로 하는 x, y 값을 찾을 수 있음. x 는 크면 클수록, y 는 0일 경우 가장 최대의 값을 반환

하지만 함수 식 자체를 알 수 없고, 단지 입력값과 반환값만 알 수 있는 상황에서 함수 반환값의 최대/ 최소값을 찾기란 매우 어려움.

→ 베이지안 최적화를 이용

베이지안 최적화는 베이지안 확률에 기반을 두고 있는 최적화 기법



베이지안 확률

- 새로운 사건의 관측이나 새로운 샘플 데이터를 기반으로 사후 확률을 개선해 나감

이와 비슷하게, 베이지안 최적화는 새로운 데이터를 입력받았을 때 최적 함수를 예측하는 사후 모델을 개선해 나가면서 최적 함수 모델을 만들어냄

▼ 대체 모델(Surrogate Model)

획득 함수로부터 최적 함수를 예측할 수 있는 입력 값을 추천 받은 뒤 이를 기반으로 최적 함수 모델을 개선해 나감

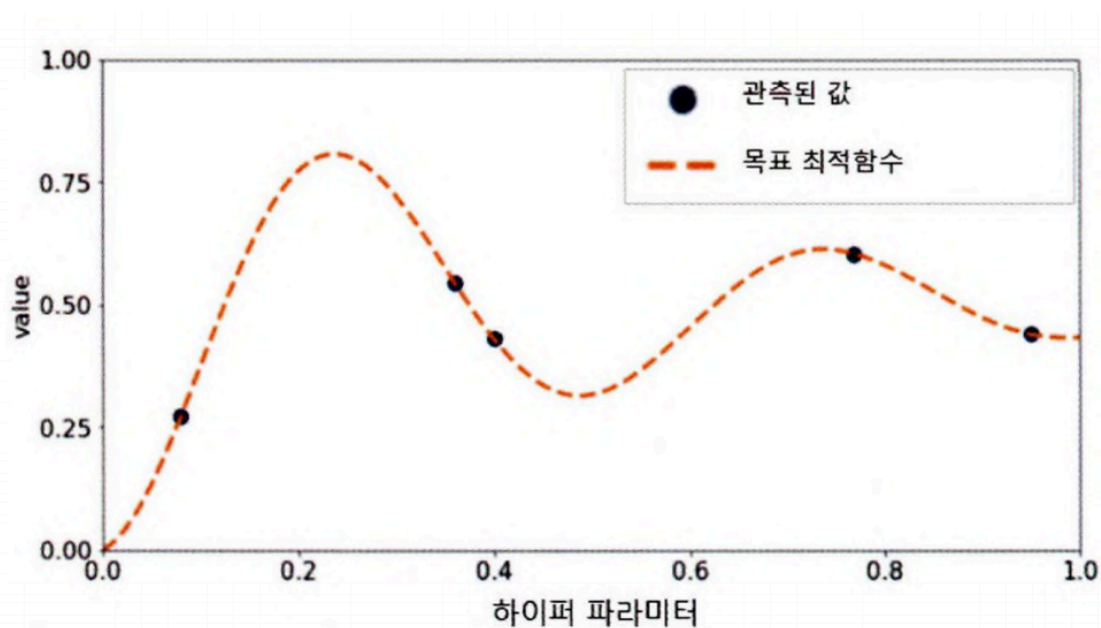
▼ 획득 함수(Acquisition Function)

개선된 대체 모델을 기반으로 최적 입력값을 계산

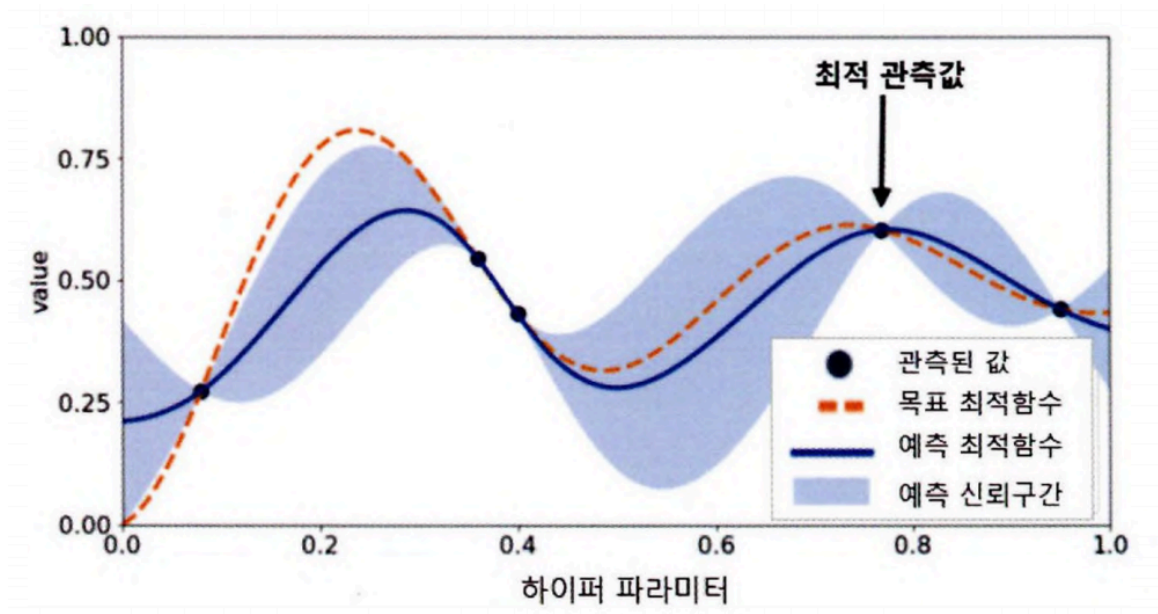
베이지안 최적화를 하이퍼 파라미터 튜닝에 사용될 때 입력값은 하이퍼 파라미터가 됨. 즉, 대체 모델은 획득 함수가 계산한 하이퍼 파라미터를 입력 받으면서 점차적으로 개선되며, 개선된 대체 모델을 기반으로 획득 함수는 더 정확한 하이퍼 파라미터를 계산할 수 있게 됨.

• 베이지안 최적화의 단계

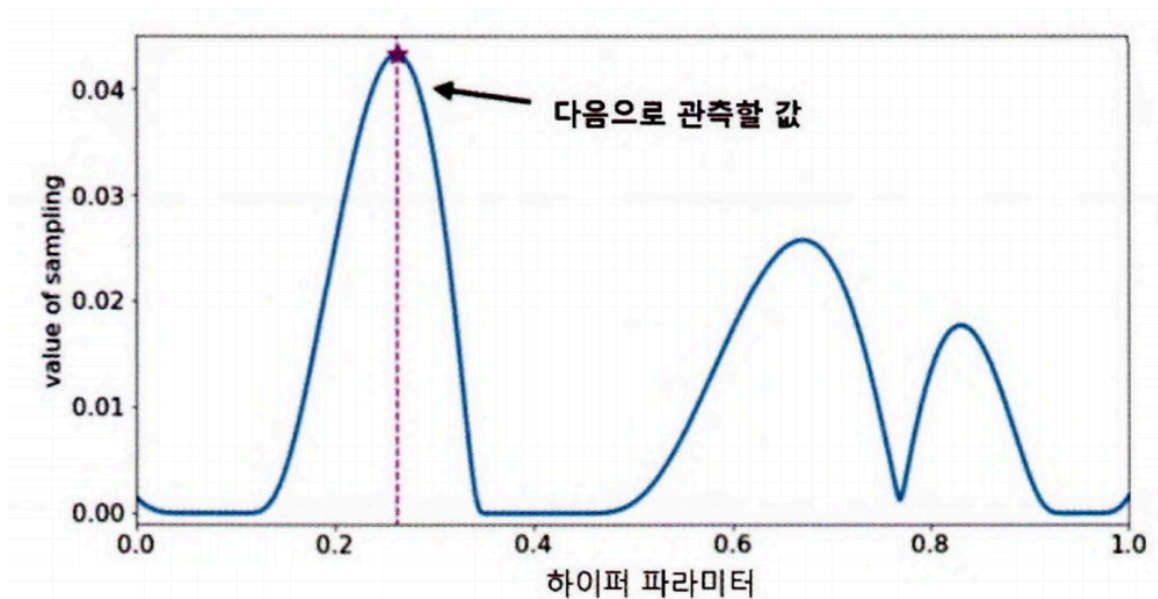
1. 최초에는 랜덤하게 하이퍼 파라미터들을 샘플링하고 성능 결과를 관측. 아래 그림에서 검은색 원은 특정 하이퍼 파라미터가 입력되었을 때 관측된 성능 지표 결과값을 뜻하며 주황색 사선은 찾아야 할 목표 최적 함수임.



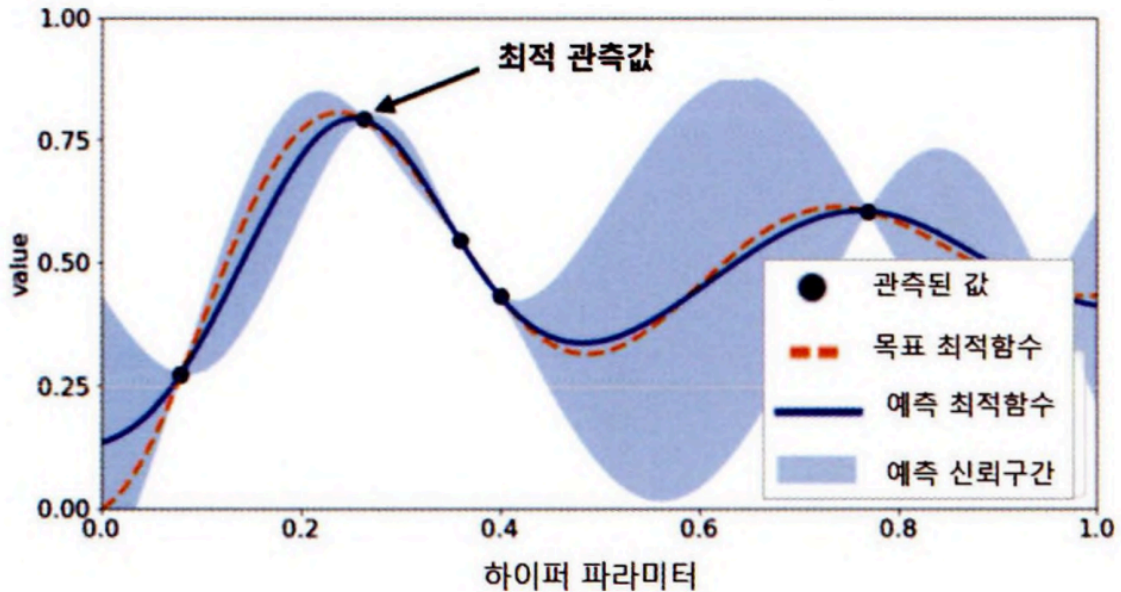
2. 관측된 값을 기반으로 대체 모델은 최적 함수를 추정. 아래 그림에서 파란색 실선은 대체 모델이 추정한 최적 함수. 옅은 파란색으로 되어 있는 영역은 예측된 함수의 신뢰 구간. 추정된 함수의 결과값 오류 편차를 의미하며 추정 함수의 불확실성을 나타냄. 최적 관측 값은 y축 value에서 가장 높은 값을 가질 때의 하이퍼 파라미터.



- 추정된 최적 함수를 기반으로 획득 함수는 다음으로 관측할 하이퍼 파라미터 값을 계산함. 획득 함수는 이전의 최적 관측값보다 더 큰 최댓값을 가질 가능성이 높은 지점을 찾아서 다음에 관측할 하이퍼 파라미터를 대체 모델에 전달함.



- 획득 함수로부터 전달된 하이퍼 파라미터를 수행하여 관측된 값을 기반으로 대체 모델은 갱신되어 다시 최적 함수를 예측 추정함.



이런 방식으로 과정3,4를 특정 횟수만큼 반복하게 되면 대체 모델의 불확실성이 개선되고 점차 정확한 최적 함수 추정이 가능하게 됨.

대체 모델은 최적 함수를 추정할 때 다양한 알고리즘을 사용할 수 있는데 일반적으로는 가우시안 프로세스(Gaussian Process)를 적용함. 그러나 HyperOpt는 가우시안 프로세스가 아닌 트리 파르젠 Estimator(TPE, Tree-structure Parzen Estimator)를 사용

HyperOpt 사용하기

베이지안 최적화를 머신러닝 모델의 하이퍼 파라미터 튜닝에 적용할 수 있게 제공되는 여러 파이썬 패키지들 중 대표적인 것은 HyperOpt, Bayesian Optimization, Optuna 등을 있음.

HyperOpt 활용 로직

- 첫째는 입력 변수명과 입력값의 검색 공간 설정
- 둘째는 목적 함수의 설정
- 마지막으로 목적 함수의 반환 최솟값을 가지는 최적 입력값을 유추

HyperOpt를 사용할 때 유의할 점은 다른 패키지와 다르게 목적 함수 반환 값의 최댓값이 아닌 최솟값을 가지는 최적 입력값을 유추한다는 것.

입력 변수명과 입력 값 검색 공간은 파이썬 딕셔너리 형태로 설정되어야 하며, 키값으로 입력 변수명, 밸류값으로 해당 입력 변수의 검색 공간이 주어짐.

```
from hyperopt import hp

search_space = {'x':hp.quniform('x', -10, 10, 1), 'y':hp.quniform('y', -15, 15, 1)}
```

입력값의 검색 공간을 제공하는 대표적인 함수는 아래와 같음. 함수 인자로 들어가는 label은 입력 변수명을 다시 적어줌. low는 최솟값, high는 최댓값이며 q는 간격

- `hp.quniform(label, low, high, q)`: label로 지정된 입력값 변수 검색 공간을 최솟값 low에서 최댓값 high까지 q의 간격을 가지고 설정.
- `hp.uniform(label, low, high)`: 최솟값 low에서 최댓값 high까지 정규 분포 형태의 검색 공간 설정
- `hp.randint(label, upper)`: 0부터 최댓값 upper까지 random한 정숫값으로 검색 공간 설정.
- `hp.loguniform(label, low, high)`: $\exp(\text{uniform}(\text{low}, \text{high}))$ 값을 반환하며, 반환 값의 log 변환 된 값은 정규 분포 형태를 가지는 검색 공간 설정.
- `hp.choice(label, options)`: 검색 값이 문자열 또는 문자열과 숫자값이 섞여 있을 경우 설정. Options는 리스트나 튜플 형태로 제공되며 `hp.choice('tree_criterion', ['gini', 'entropy'])`과 같이 설정하면 입력 변수 `tree_criterion`의 값을 'gini'와 'entropy'로 설정하여 입력함.

목적함수는 반드시 변숫값과 검색 공간을 가지는 딕셔너리를 인자로 받고, 특정 값을 반환하는 구조로 만들어져야 함.

목적함수의 반환값은 숫자형 단일값 외에도 딕셔너리 형태로 반환할 수 있음

```
from hyperopt import STATUS_OK

def objective_func(search_space):
    x = search_space['x']
    y = search_space['y']
    retval = x**2 - 20*y
```

```
return retval
```

목적 함수의 반환값이 최소가 될 수 있는 최적의 입력값을 베이지안 최적화 기법에 기반해서 찾아야 함. HyperOpt는 이러한 기능을 수행할 수 있도록 `fmin(objective, space, algo, max_evals, trials)` 함수를 제공함. `fmin()` 함수의 주요인자는 아래와 같음

- `fn`: 위에서 생성한 `objective_func`와 같은 목적 함수입니다.
- `space`: 위에서 생성한 `search_space`와 같은 검색 공간 딕셔너리입니다.
- `algo`: 베이지안 최적화 적용 알고리즘 입니다. 기본적으로 `tpe.suggest`이며 이는 HyperOpt의 기본 최적화 알고리즘인 TPE(Tree of Parzen Estimator)를 의미합니다.
- `max_evals`: 최적 입력값을 찾기 위한 입력값 시도 횟수입니다.
- `trials`: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는 데 사용됩니다. Trials 클래스를 객체로 생성한 변수명을 입력합니다.
- `rstate`: `fmin()`을 수행할 때마다 동일한 결괏값을 가질 수 있도록 설정하는 랜덤 시드(seed) 값입니다.

HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

적용해야 할 하이퍼 파라미터와 검색 공간을 설정하고, 목적 함수에서 XGBoost를 학습 후에 예측 성능 결과를 반환 값으로 설정함. 그리고 `fmin()` 함수에서 목적 함수를 하이퍼 파라미터 검색 공간의 입력값들을 사용하여 최적의 예측 성능 결과를 반환하는 최적 입력값들을 결정함.

특정 하이퍼 파라미터들은 정숫값만 입력 받는데 HyperOpt는 입력값과 반환값이 모두 실수형이기 때문에 하이퍼 파라미터 입력 시 형변환을 해줘함을 주의

HyperOpt의 목적 함수는 최솟값을 반환할 수 있도록 최적화해야 하기 때문에 성능 값이 클수록 좋은 성능 지표일 경우 -1을 곱해 줘야 한다는 점을 주의

10. 분류 실습 - 캐글 신용카드 사기 검출



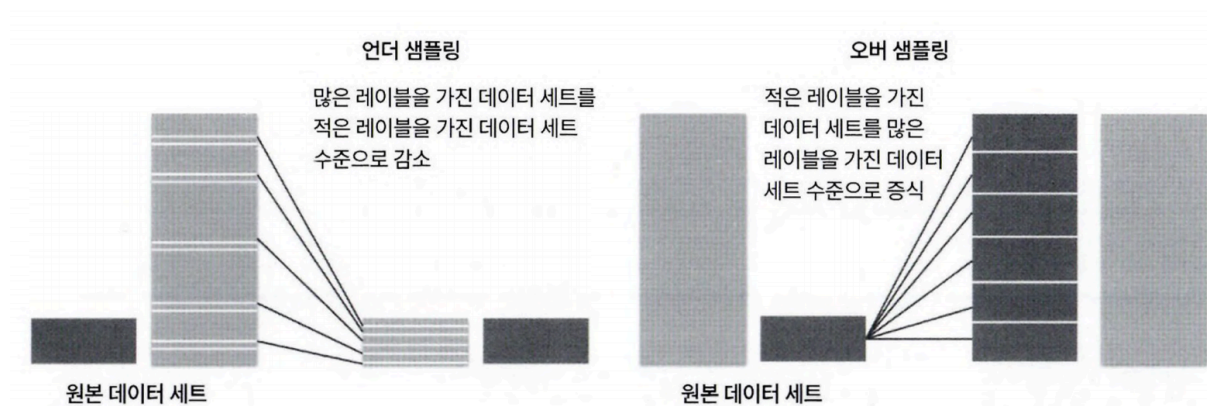
해당 데이터 세트의 레이블인 Class 속성은 매우 불균형한 분포를 가지고 있음.
Class는 0과 1로 분류되는데 0이 사기가 아닌 정상적인 신용카드 트랜잭션 데이터, 1은 신용카드 사기 트랜잭션을 의미.
전체 데이터의 약 0.172%만이 레이블 값이 1, 즉 사기 트랜잭션

언더 샘플링과 오버 샘플링의 이해

레이블이 불균형한 분포를 가진 데이터 세트를 학습시킬 때 예측 성능의 문제가 발생할 수 있는데, 이는 이상 레이블을 가지는 데이터 건수가 정상 레이블을 가진 데이터 건수에 비해 너무 적기 때문에 발생함.

즉, 이상 레이블을 가지는 데이터 건수는 매우 적기 때문에 제대로 다양한 유형을 학습하지 못하는 반면에 정상 레이블을 가지는 데이터 건수는 매우 많기 때문에 일방적으로 정상 레이블로 치우친 학습을 수행해 제대로 된 이상 데이터 검출이 어려워지기 쉬움

지도 학습에서 극도로 불균형한 레이블 값 분포로 인한 문제점을 해결하기 위해서는 적절한 학습 데이터를 확보하는 방안이 필요한데, 대표적으로 **오버 샘플링(Oversampling)**과 **언더 샘플링(Undersampling)** 방법이 있으며, 오버 샘플링 방식이 예측 성능상 조금 유리한 경우가 많아 상대적으로 더 많이 사용됨.



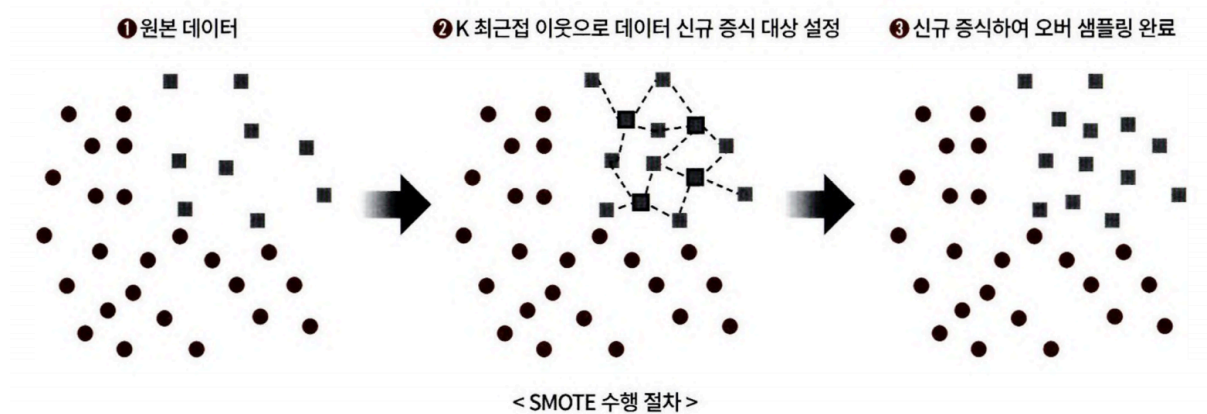
언더 샘플링

- 많은 데이터 세트를 적은 데이터 세트 수준으로 감소시키는 방식
- 정상 레이블 데이터를 이상 레이블 데이터 수준으로 줄여 버린 상태에서 학습을 수행하면 과도하게 정상 레이블로 학습/예측하는 부작용을 개선할 수 있지만, 너무 많은 정상

레이블 데이터를 감소시켜서 정상 레이블의 경우 제대로 된 학습을 수행할 수 없는 문제가 발생할 수도 있으므로 유의해야 함.

오버 샘플링

- 이상 데이터와 같이 적은 데이터 세트를 증식하여 학습을 위한 충분한 데이터를 확보하는 방법
- 동일한 데이터를 단순히 증식하는 방법은 과적합이 되기 때문에 의미가 없으므로 원본 데이터의 피쳐 값들을 아주 약간만 변경하여 증식함
- 대표적으로 **SMOTE(Synthetic Minority Over-sampling Technique)** 방법이 있음. SMOTE는 적은 데이터 세트에 있는 개별 데이터들의 K 최근접 이웃(K Nearest Neighbor)을 찾아서 이 데이터와 K개 이웃들의 차이를 일정 값으로 만들어서 기존 데이터와 약간 차이가 나는 새로운 데이터들을 생성하는 방식



이상치 데이터 제거 후 모델 학습/예측/평가

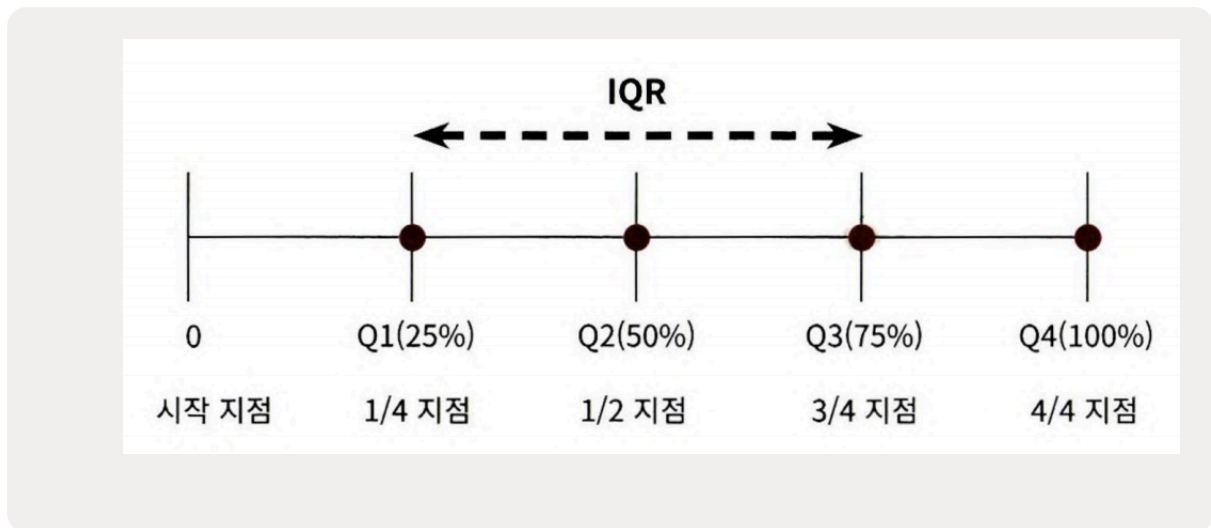
이상치 데이터(Outlier)는 전체 데이터의 패턴에서 벗어난 이상 값을 가진 데이터이다. 이상치로 인해 머신러닝 모델의 성능에 영향을 받는 경우가 발생하기 쉬움

이상치를 찾는 방법 중 대표적인 것으로는 IQR(Inter Quantile Range)가 있음. IQR은 사분위(Quantile) 값의 편차를 이용하는 기법으로 흔히 박스 플롯 방식으로 시각화가 가능

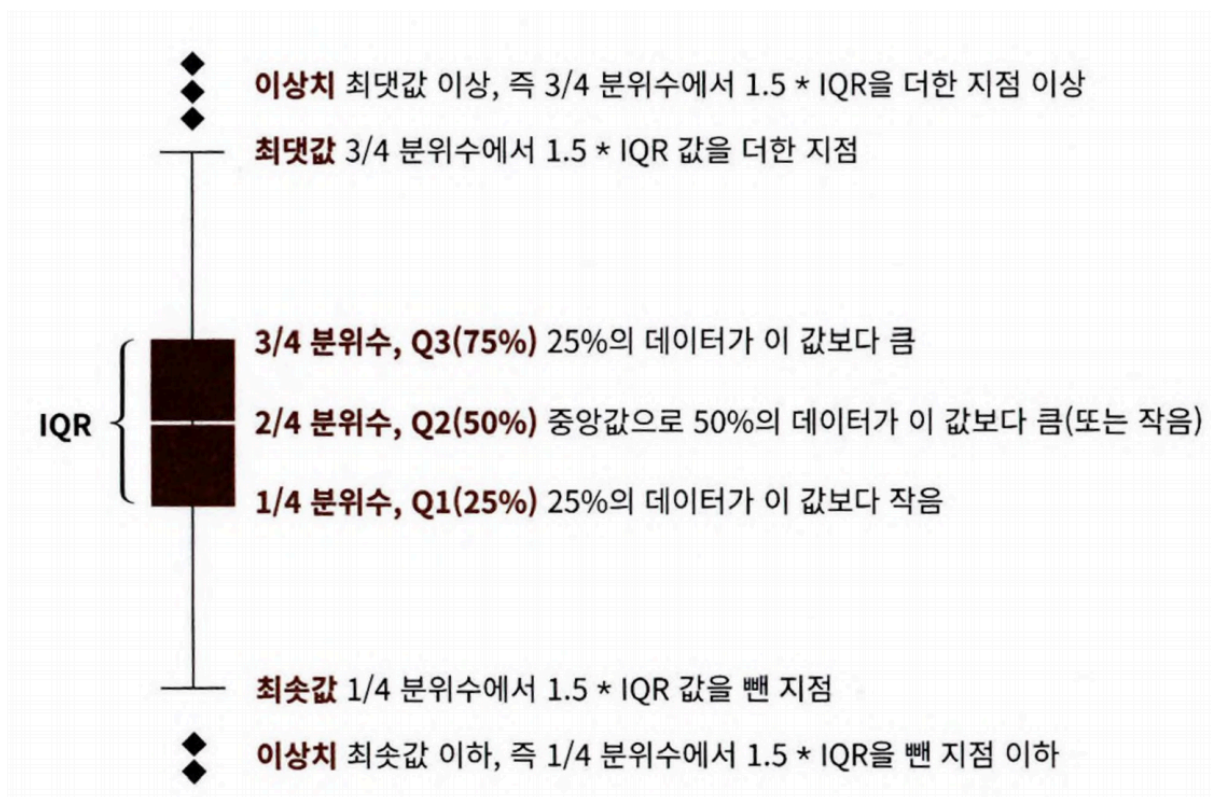


사분위(Quantile)

사분위는 전체 데이터를 값이 높은 순으로 정렬하고, 이를 1/4(25%)씩으로 구간을 분할하는 것을 지칭함.



IQR을 이용해 이상치 데이터를 검출하는 방식은 보통 IQR에 1.5를 곱해서 생성된 범위를 이용해 최댓값과 최솟값을 결정한 뒤 최댓값을 초과하거나 최솟값에 미달하는 데이터를 이상치로 간주하는 것이다



정리

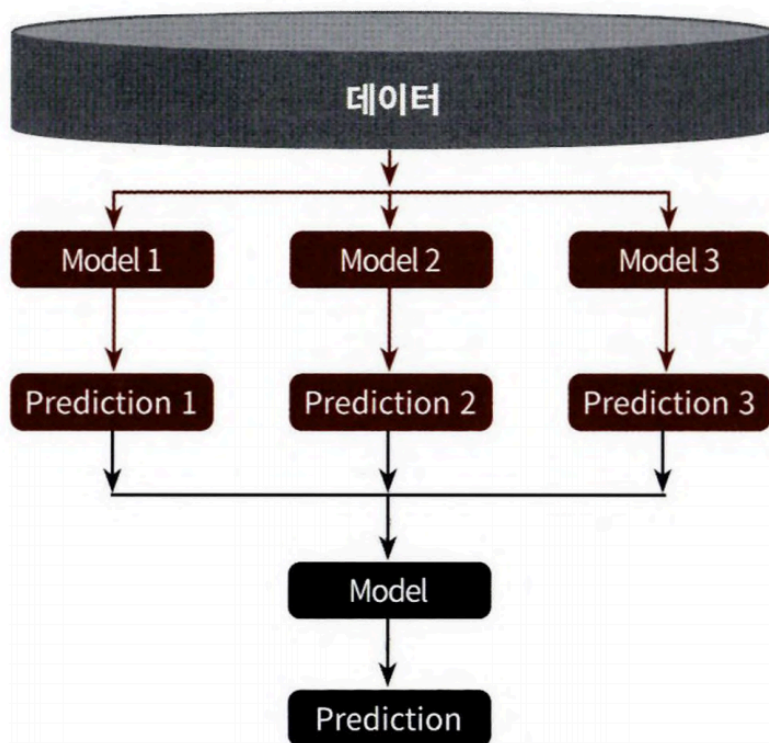
데이터 가공 유형	머신러닝 알고리즘	평가 지표		
		정밀도	재현율	ROC-AUC
원본 데이터 가공 없음	로지스틱 회귀	0.8679	0.6216	0.9702
	LightGBM	0.9573	0.7568	0.9790
데이터 로그 변환	로지스틱 회귀	0.8812	0.6014	0.9727
	LightGBM	0.9576	0.7635	0.9796
이상치 데이터 제거	로지스틱 회귀	0.8750	0.6712	0.9743
	LightGBM	0.9603	0.8288	0.9780
SMOTE 오버 샘플링	로지스틱 회귀	0.0542	0.9247	0.9737
	LightGBM	0.9118	0.8493	0.9814

11. 스택킹 앙상블

스택킹은 개별적인 여러 알고리즘을 서로 결합해 예측 결과를 도출한다는 점에서 앞에 소개한 배깅 및 부스팅과 공통점을 갖고 있음.

하지만 가장 큰 차이점은 개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측을 수행한다는 점. 즉, 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종 학습을 수행하고 테스트 데이터를 기반으로 다시 최종 예측을 수행하는 방식

스택킹 모델은 두 종류의 모델이 필요함 **1. 개별적인 기반 모델, 2. 이 개별 기반 모델의 예측 데이터를 학습 데이터로 만들어서 학습하는 최종 메타 모델** 스택킹 모델의 핵심은 여러 개별 모델의 예측 데이터를 각각 스택킹 형태로 결합해 최종 메타 모델의 학습용 피쳐 데이터 세트와 테스트용 데이터 세트를 만드는 것

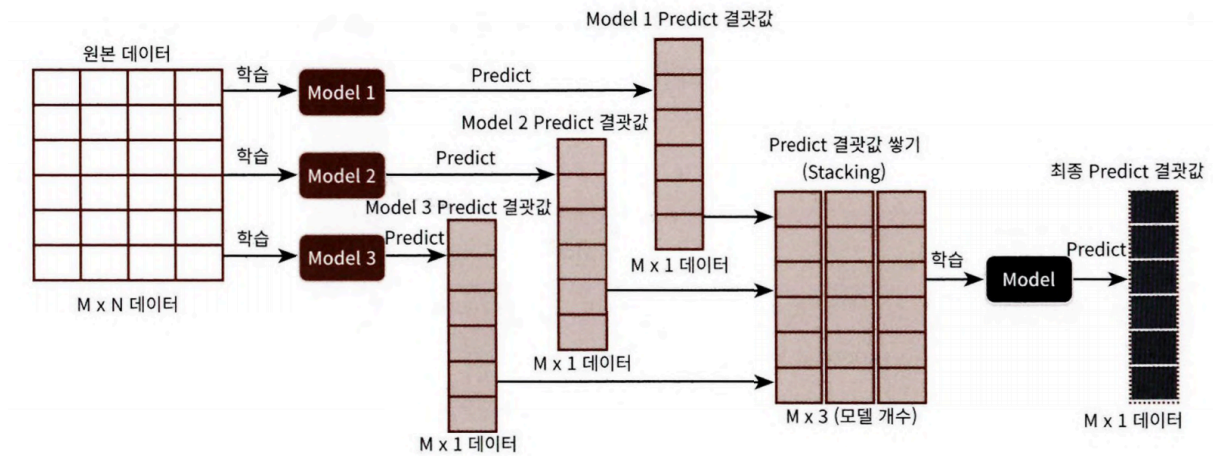


M개의 로우, N개의 피처(칼럼)를 가진 데이터 세트에 스택킹 앙상블을 적용한다고 가정하자.

그리고 학습에 사용할 ML 알고리즘 모델은 모두 3개

먼저 모델별로 각각 학습을 시킨 뒤 예측을 수행하면 각각 M개의 로우를 가진 1개의 레이블 값을 도출할 것

모델별로 도출된 예측 레이블 값을 다시 합해서(스태킹) 새로운 데이터 세트를 만들고 이렇게 스택킹된 데이터 세트에 대해 최종 모델을 적용해 최종 예측을 하는 것이 스택킹 앙상블 모델

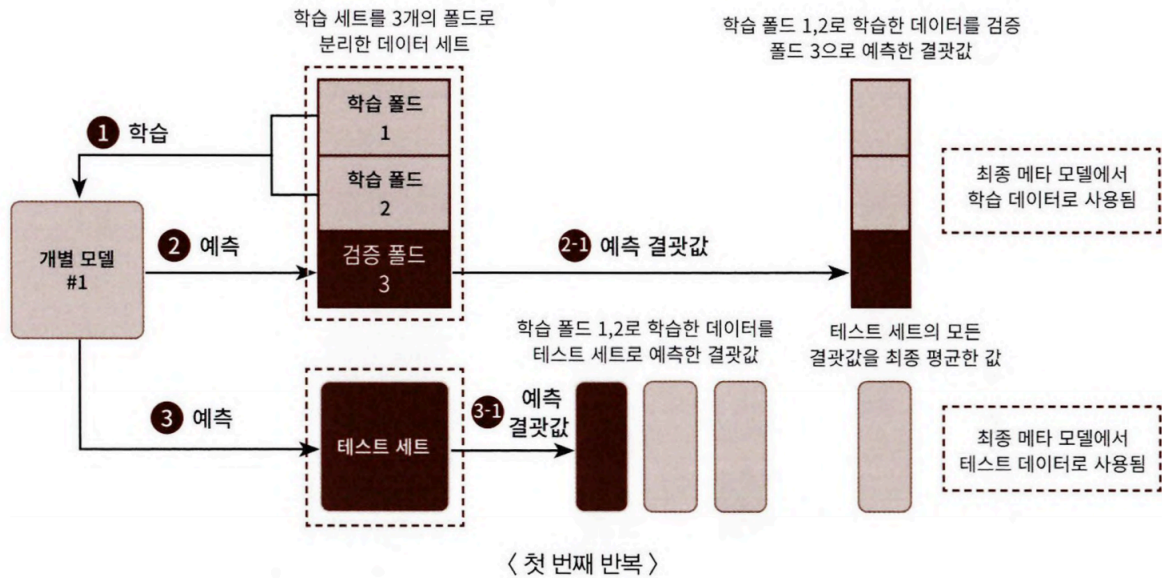


CV 세트 기반의 스택킹

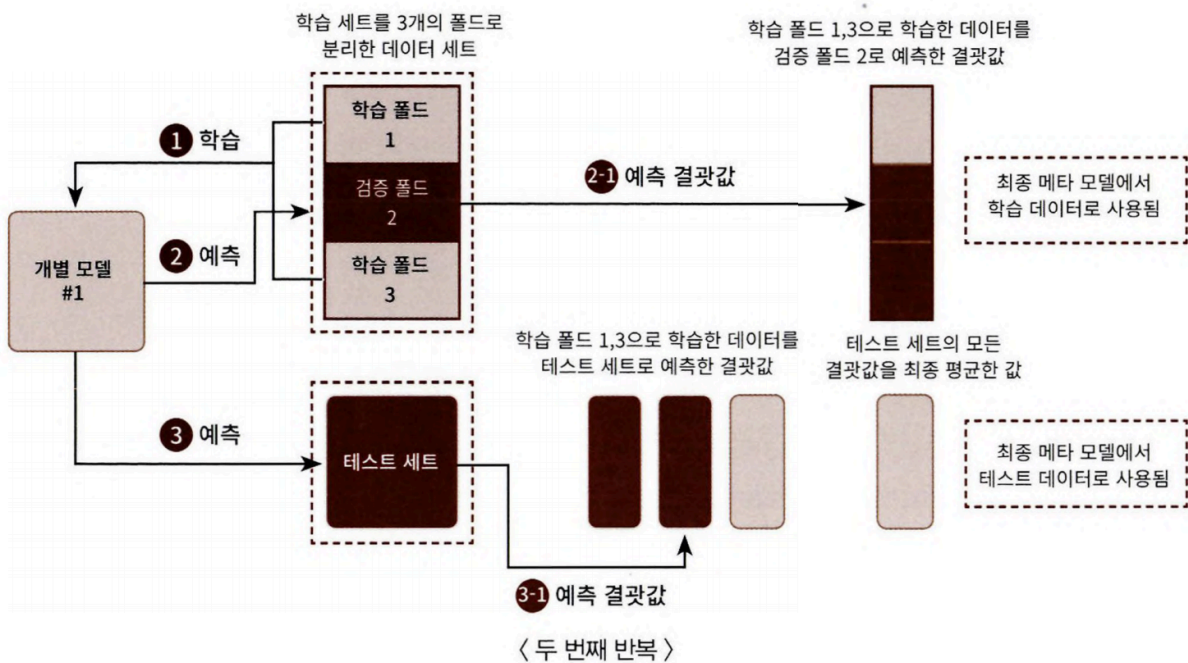
CV 세트 기반의 스택킹 모델은 과적합을 개선하기 위해 최종 메타 모델을 위한 데이터 세트를 만들 때 교차 검증 기반으로 예측된 결과 데이터 세트를 이용

CV 세트 기반의 스택킹은 이에 대한 개선을 위해 개별 모델들이 각각 교차 검증으로 메타 모델을 위한 학습용 스택킹 데이터 생성과 예측을 위한 테스트용 스택킹 데이터를 생성한 뒤 이를 기반으로 메타 모델이 학습과 예측을 수행

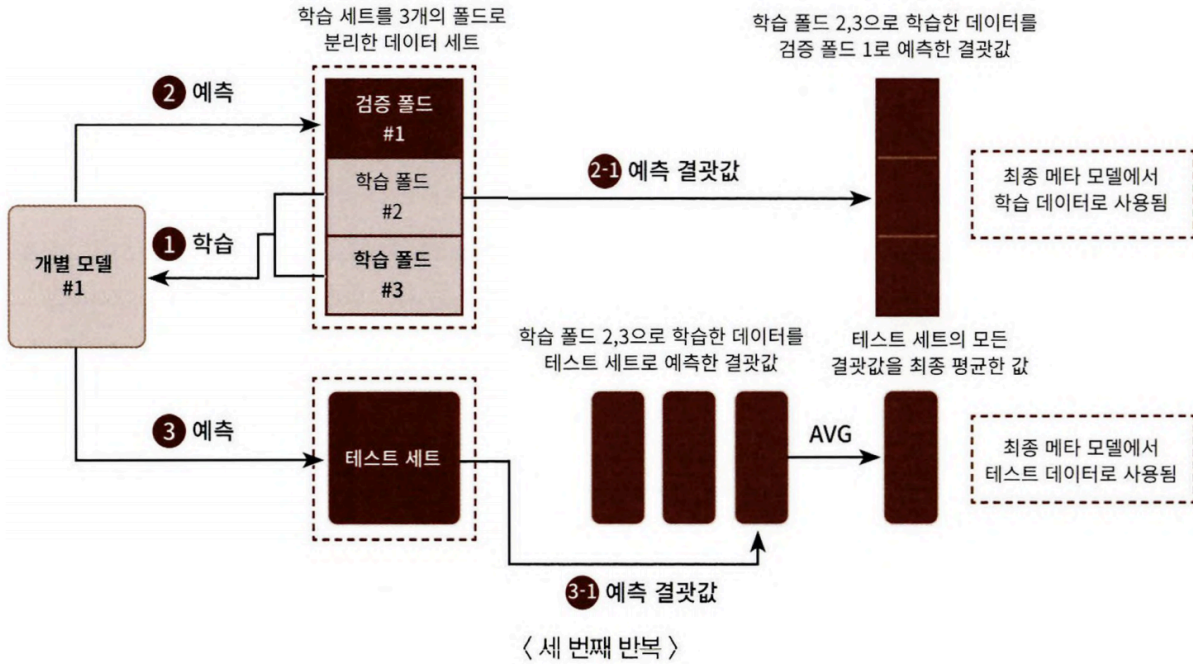
- **Step1:** 각 모델별로 원본 학습/테스트 데이터를 예측한 결과 값을 기반으로 메타 모델을 위한 학습용/테스트용 데이터를 생성
- **Step2:** Step1에서 개별 모델들이 생성한 학습용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 학습할 최종 학습용 데이터 세트를 생성. 마찬가지로 각 모델들이 생성한 테스트용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 예측할 최종 테스트 데이터 세트를 생성함. 메타 모델은 최종적으로 생성된 학습 데이터 세트와 원본 학습 데이터의 레이블 데이터를 기반으로 학습한 뒤, 최종적으로 생성된 테스트 데이터 세트를 예측하고, 원본 테스트 데이터의 레이블 데이터를 기반으로 평가함



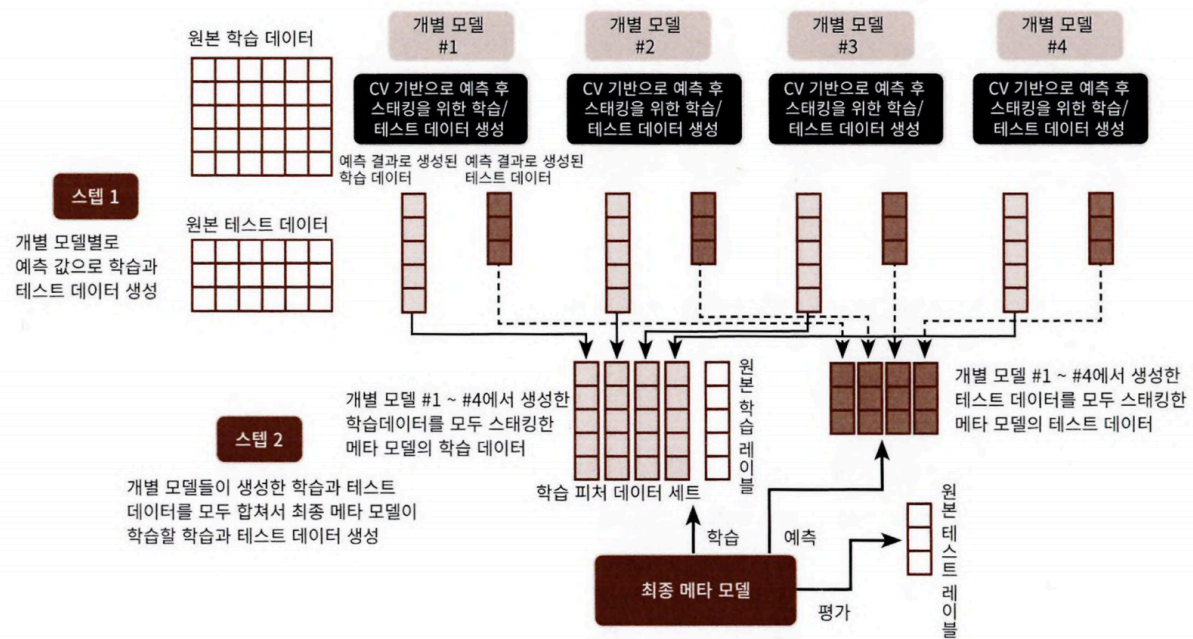
↳ 3개의 폴드만큼 반복을 수행하면서 스택킹 데이터를 생성하는 첫 번째 반복



↳ 스택킹 데이터를 생성하는 두 번째 반복. 폴드 내의 학습용 데이터 세트를 변경하고 첫 번째 그림과 동일한 작업을 수행함



ㄴ 세 번째 반복을 수행하면서 폴드 내의 학습용 데이터 세트가 변경됨. 세 번째 반복을 완료하면 첫 번째, 두 번째, 세 번째 반복을 수행하면서 만들어진 폴드별 예측 데이터를 합하여 메타 모델에서 사용될 학습 데이터를 만들게 됨. 마찬가지로 첫 번째, 두 번째, 세 번째 반복을 수행하면서 학습 폴드 데이터로 학습된 개별 모델이 원본 테스트 세트로 예측한 결과값을 최종 평균하여 메타 모델에서 사용될 테스트 데이터를 만들게 됨.



ㄴ 스텝 2를 포함한 CV 기반의 스택킹 모델 전체를 도식화한 그림

정리

스태킹을 이루는 모델은 최적으로 파라미터를 튜닝한 상태에서 스태킹 모델을 만드는 것이 일반적

스태킹 모델은 분류뿐만 아니라 회귀에도 적용이 가능